

Memory Layer Extension

Installation & Setup Guide

Version 1.0

Built by Swati

GitHub Repository:

<https://github.com/gitswati-27/memory-layer-extension>

Table of Contents

1. Introduction
 2. What is Memory Layer Extension?
 3. Architecture Overview
 4. Hosted vs Self-Hosted Setup
 5. Prerequisites
 6. Downloading the Project
 7. Creating a Neon Database
 8. Generating a Gemini API Key
 9. Generating a Groq API Key
 10. Configuring Environment Variables
 11. Installing Dependencies
 12. Database Setup with Prisma
 13. Running the Backend
 14. Building the Extension
 15. Loading the Extension in Chrome
 16. Using Memory Layer Extension
 17. Troubleshooting
 18. FAQ
-

1. Introduction

Welcome to Memory Layer Extension.

Memory Layer Extension is an AI-powered Chrome extension that enables users to:

- Save webpages
- Organize content into collections
- Search memories using keywords
- Perform semantic search using vector embeddings
- Chat with saved knowledge using Retrieval-Augmented Generation (RAG)

This guide explains everything required to install and configure your own instance of Memory Layer Extension.

2. What is Memory Layer Extension?

Most valuable information discovered online eventually disappears into:

- Browser history
- Bookmark folders
- Open tabs
- Note-taking applications

Memory Layer Extension transforms saved webpages into a searchable AI-powered knowledge base.

Instead of remembering where information came from, users can simply ask questions and retrieve relevant information instantly.

3. Architecture Overview

The system consists of:

Chrome Extension

↓

Express Backend

↓

Neon PostgreSQL Database

↓

pgvector

↓

Gemini Embeddings

↓

Groq Large Language Model

The Chrome extension extracts webpage content and sends it to the backend.

The backend stores content, generates embeddings, performs semantic retrieval, and powers AI-assisted conversations.

4. Hosted vs Self-Hosted Setup

Hosted Mode

Hosted Mode uses the public backend deployment.

Backend URL:

<https://memory-layer-extension.onrender.com>

This mode is suitable for:

- Quick testing
 - Demonstrations
 - Exploring the extension
-

Self-Hosted Mode

Self-hosted mode provides complete control over:

- Database
- API usage
- Embedding generation
- AI model usage

This guide primarily focuses on self-hosting.

5. Prerequisites

Before beginning installation ensure that the following are available:

Required

- Google Chrome
- Node.js (Latest LTS recommended)
- npm
- Git

Accounts

- GitHub
 - Neon
 - Google AI Studio
 - Groq
-

6. Downloading the Project

Option 1 — Clone Repository

Open a terminal and run:

```
git clone https://github.com/gitswati-27/memory-layer-extension.git
```

Navigate into the project:

```
cd memory-layer-extension
```

Option 2 — Download ZIP

1. Visit the GitHub repository.
 2. Click Code.
 3. Click Download ZIP.
 4. Extract the downloaded archive.
-

7. Creating a Neon Database

Neon provides a free PostgreSQL database.

Step 1

Create an account at:

<https://neon.tech>

Step 2

Create a new project.

Example:

```
memory-layer-extension
```

Step 3

Create a database.

Example:

```
neondb
```

Step 4

Copy the PostgreSQL connection string.

Example:

```
postgresql://username:password@host/database
```

Keep this value for the environment configuration section.

8. Generating a Gemini API Key

Gemini is used for generating embeddings.

Step 1

Visit:

<https://aistudio.google.com>

Step 2

Sign in.

Step 3

Create a new API key.

Step 4

Copy and securely store the key.

Example:

```
GEMINI_API_KEY=your_key_here
```

9. Generating a Groq API Key

Groq powers the AI assistant.

Step 1

Visit:

<https://console.groq.com>

Step 2

Create an account.

Step 3

Navigate to API Keys.

Step 4

Generate a new API key.

Step 5

Copy the generated key.

Example:

```
GROQ_API_KEY=your_key_here
```

10. Configuring Environment Variables

Navigate to the backend directory.

Create a file named:

```
.env
```

Add the following:

```
DATABASE_URL=your_neon_database_url
```

```
GEMINI_API_KEY=your_gemini_api_key
```

```
GROQ_API_KEY=your_groq_api_key
```

Verify that all values have been copied correctly.

11. Installing Dependencies

Install frontend dependencies:

```
npm install
```

Install backend dependencies:

```
cd backend  
npm install
```

12. Database Setup with Prisma

Apply Prisma migrations:

```
npx prisma migrate deploy
```

Generate Prisma Client:

```
npx prisma generate
```

This creates all required database tables.

13. Running the Backend

Navigate to the backend directory.

Start the backend:

```
npm start
```

The server should start successfully.

Example:

```
Server running on port 5000
```

14. Building the Extension

Return to the extension root.

Build the extension:

```
npm run build
```

A folder named:

```
dist
```

will be generated.

15. Loading the Extension in Chrome

Open:

`chrome://extensions`

Enable:

Developer Mode

Click:

Load Unpacked

Select:

`dist`

The extension should now appear in Chrome.

Pin the extension for easier access.

16. Using Memory Layer Extension

Saving Pages

1. Open a webpage.
 2. Open the extension popup.
 3. Select a collection.
 4. Click Save Current Page.
-

Searching Memories

1. Open View All Memories.
 2. Enter a search query.
 3. Browse matching results.
-

AI Assistant

1. Open View All Memories.
2. Click the AI Assistant button.
3. Ask questions about saved content.

Examples:

What have I saved about React?

Summarize my system design notes.

Explain caching concepts from my saved resources.

17. Troubleshooting

Extension Does Not Load

Verify:

- Developer Mode is enabled
 - dist folder exists
 - Build completed successfully
-

Database Connection Failed

Verify:

- DATABASE_URL is correct
 - Neon database is running
 - Network access is available
-

Gemini Errors

Verify:

- API key is valid
 - API key is present in .env
-

Groq Errors

Verify:

- API key is valid
 - Usage limits have not been exceeded
-

Semantic Search Returns Weak Results

This is expected when very few memories exist.

Semantic retrieval improves as additional content is saved.

18. FAQ

Do I need my own API keys?

Yes.

Self-hosted deployments require both Gemini and Groq API keys.

Why use Neon?

Neon provides a managed PostgreSQL database with a generous free tier.

Why is semantic search useful?

Semantic search retrieves information based on meaning rather than exact keyword matches.

Why does the first request sometimes take longer?

Render free-tier deployments may enter a sleep state when inactive.

The first request may take additional time while the service wakes up.

Is this available on the Chrome Web Store?

Not currently.

The extension must be loaded manually using Chrome Developer Mode.

Conclusion

Thank you for using Memory Layer Extension.

Memory Layer Extension demonstrates how modern AI systems can transform saved information into an intelligent, searchable knowledge base.

Built by Swati

GitHub:

<https://github.com/gitswati-27/memory-layer-extension>